Phase 1 Report: Data Collection and Tokenizer Construction

Building and Analyzing Monolingual Transformer LMs in Two Indian Languages

Erva Rishitha (2025201027)

August 21, 2026

Abstract

This report covers Phase 1 of my individual project: building independent monolingual text corpora and SentencePiece BPE tokenizers for two Indian languages of different resource tiers – Telugu as the higher-resource language (Model H) and Nepali as the lower-resource language (Model L). I have described how I collected 500M tokens per language from a mix of downloaded corpora (AI4Bharat Sangraha) and manually-collected sources (web crawling, Wikimedia dumps and APIs, and OCR of textbooks and archived documents), the cleaning and deduplication pipeline I built, and the tokenizer ablation study that led me to a final vocabulary size of 12,000 for both languages. Both corpora clear the required $\geq$20% manual-data threshold (Nepali at 20.22%, Telugu at 28.12%) and both exceed the $\sim$500M token target.

Data & Code Availability.

- **OneDrive (primary source):** link – raw, cleaned/deduplicated, and split corpora plus tokenizer files for both languages.
- **Google Drive:** link – OCR source textbooks/archive PDFs and raw Colab/Kaggle notebook run outputs.

1 Language Selection

Model H (higher-resource): Telugu. I picked Telugu mainly because it has strong existing support in AI4Bharat's Sangraha corpus (both a verified and unverified `tel` subset), a full Wikimedia ecosystem (Wikipedia, Wikisource, Wikibooks, Wiktionary all exist in Telugu), and a large number of active Telugu news sites I could crawl. That combination made it realistic to actually hit the $\sim$500M token target without having to lean too hard on any single source and it will be easier for me to understand as Telugu is my mother tongue.

Model L (lower-resource): Nepali. Nepali is on the allowed lower-resource list. Sangraha does include a Nepali subset, but it is explicitly flagged as low-resource within that dataset – I confirmed this myself when the `verified/nep` split alone was enough to hit my download target, and I never even needed the fallback sources (IndicCorpV2, CC-100) I had built into the downloader in case Sangraha ran dry. Nepali also uses the Devanagari script, which gave me a nice point of contrast against Telugu's abugida when I got to the tokenizer stage – fertility and compression behave differently across the two scripts.

2 Data Collection

Every corpus in this project is split into two tiers that get tracked separately all the way through the pipeline:

- **Downloaded tier**: streamed from AI4Bharat Sangraha via the `datasets` library, tagged `sangraha_downloaded`.
- **Manual tier**: everything I collected myself – web crawling (news sites, government portals, literature sites, Wikimedia dumps/APIs, Internet Archive) and OCR of scanned textbooks/archived PDFs.

This split matters because the brief requires at least 20% of the final training tokens to come from the manual tier, and I wanted that number visible at every stage rather than discovering it only at the end.

2.1 Downloaded tier

2.1.1 Nepali

`nepali_sangraha.py` streams `ai4bharat/sangraha` in streaming mode (so nothing has to fit in memory) with a target of 400M words. Because Nepali is flagged low-resource in Sangraha, I built the script to fall through an ordered list of sources once a given source runs dry: `sangraha verified/nep` → `sangraha unverified/nep` → `IndicCorpV2 npi_Deva` → `CC-100 ne`. In practice, `verified/nep` alone (1,334,775 documents) reached the target and the fallback sources were never triggered – confirmed from the resumable progress file, which shows zero items consumed on all three fallback sources.

2.1.2 Telugu

`telugu_sangraha.py` follows the same streaming/checkpointing pattern but only needs `sangraha verified/tel` and `unverified/tel` (target 500M words) – Telugu did not need the extra fallback sources Nepali required.

2.1.3 Trimming

Both raw downloaded dumps came in larger than I wanted to carry through the rest of the pipeline, so `0_trim_*.py` trims each down to a 290M-word budget. This is done at the block level (each document is wrapped in `<<SRC:...>>...<<END>>` markers), so trimming never cuts a document in half – it stops only on a complete block boundary once the word budget is hit.

2.2 Manual tier

This is the part of the pipeline that actually took the most real-world time and had the most things go wrong along the way, so I want to walk through what each source contributed rather than just list them.

2.2.1 Web crawling

Both languages use the same crawler design: an ordered set of cells targeting Wikimedia XML dumps, then a large batch of news/government sitemap and RSS feeds crawled with per-domain rate limiting, then a live Wikimedia API pass as a fallback/supplement, then a small BFS crawl over a few literature sites, then a deep sweep of Internet Archive text items filtered by language/subject. Everything writes to one running text file tagged with `<<SRC:...>>` markers, and every stage checkpoints (`crawled_urls.json`, `failed_urls.json`) so an interrupted run resumes instead of starting over – this mattered a lot in practice since I ran these crawls across local, Colab, and Kaggle sessions at different points and none of those environments guarantee a session stays alive.

Telugu's news crawl was the single biggest manual contributor by a wide margin: one source alone (`telugu.greatandhra.com`) contributed over 76,000 saved pages in a single run, and `eenadu.net` added another ∼4,200. Nepali's manual tier is more evenly spread across 34 news/government sources rather than dominated by one or two sites.

2.2.2 OCR

Textbook and archived-document OCR ran on Google Colab using PyMuPDF (to rasterize PDF pages) and EasyOCR (GPU-accelerated OCR) for both languages, plus a separate Internet-Archive-sourced archive OCR pass for Telugu. Both OCR notebooks only commit a book to the output file after every page in that book has been processed successfully – if a session dies mid-book, that book's partial output is simply discarded and the checkpoint file doesn't mark it done, so it gets retried cleanly on the next run instead of leaving a half-written, potentially duplicated book in the corpus.

3 Preprocessing and Cleaning

Both languages go through an identical preprocessing pipeline (separate scripts, same logic), so I'll describe it once.

3.1 Normalization and filtering (`1_*_preprocess_fulldata.py`)

For every raw dump, I stream it line by line (never loading the whole file into memory) and, per document:

1. HTML-unescape and Unicode-normalize to NFC.
2. Strip known log/progress-bar artifacts that leaked into some of the raw dumps (things like `"Downloading:"`, `"IncompleteRead"`, progress-counter lines) – these came from the crawler's own `tqdm` output occasionally getting mixed into a text file during an early, less careful version of the pipeline.
3. Strip URLs, filesystem paths, and residual wiki markup (`{{...}}`, `[[...]]` templates, table-syntax leftovers).
4. Apply a per-source-type line filter: a minimum character length and a minimum script-character ratio (Devanagari-vs-Latin for Nepali, Telugu-vs-Latin for Telugu). OCR text gets a stricter threshold (0.60–0.75 depending on source) than clean web text (0.45), since OCR is noisier and more prone to stray Latin-character misreads.

5. Drop consecutive duplicate lines within a document (a common OCR/scrape artifact).

6. Drop the whole document if it ends up under 30 words after cleaning.

Each surviving document is written out as one JSON line carrying its cleaned text plus metadata: source tier (manual/downloaded), source type, source file, original `<<SRC:...>>` tag, domain/URL/subset where applicable, and word count.

3.2 Deduplication

3.2.1 Exact deduplication (`2_*_exact_deduplication.py`)

SHA-256 hashing of whitespace-normalized text, run in a single streaming pass with one shared hash set across both tiers. Manual documents are processed first, so a manual document always wins a collision – if a downloaded document turns out to be an exact duplicate of something already kept from the manual tier, the downloaded copy is the one dropped. This was a deliberate choice: since the manual tier is the scarcer, harder-won resource, I didn't want deduplication logic to accidentally end up protecting the wrong side.

3.2.2 Near deduplication (`3_*_near_deduplication.py`)

MinHash + LSH (128 permutations, 5-word shingles, Jaccard similarity threshold 0.85), using the `datasketch` library. The same manual-first priority rule applies here, and it's enforced structurally rather than just by ordering: manual and downloaded documents are streamed through *one shared* `MinHashLSH` index, manual first. By the time the downloaded pass starts, every manual document's MinHash is already in the index, so a near-duplicate downloaded document gets dropped whether it collides with an earlier downloaded document *or* with a manual one – genuine cross-tier near-duplicate protection, not just within-tier.

4 Dataset Statistics

4.1 Nepali (Model L)

Table 1: Nepali – overall corpus statistics

Metric	Value
Total documents	1,186,541
Total words	364,079,606
Total characters	2,358,860,934
Total tokens	**526,615,295**
Fertility (avg. chars/token)	4.4793
UNK rate (val)	0.0000%
UNK rate (test)	0.0000%
Manual tokens	106,471,772 (20.22%)
Downloaded tokens	420,143,523 (79.78%)

Table 2: Nepali – per-split breakdown

Split	Docs	Tokens	Manual %
train	1,174,677	521,349,099	20.23%
val	5,932	2,677,101	19.90%
test	5,932	2,589,095	18.90%

Table 3: Nepali – per-source breakdown

Source	Docs	Tokens	% of total
Sangraha / AI4Bharat (downloaded)	964,537	420,143,523	79.78%
News websites (scraped)	183,247	81,318,955	15.44%
Government portals (scraped)	35,115	17,112,310	3.25%
OCR – textbooks (manual)	134	5,365,521	1.02%
Literature websites (scraped)	1,497	1,618,851	0.31%
Wikipedia API	1,233	508,113	0.10%
Wikibooks API	67	441,077	0.08%
Wiktionary API	703	70,505	0.01%
Archive OCR	8	36,440	0.01%

4.2 Telugu (Model H)

4.3 Manual-data compliance

Both corpora clear the $\geq$20% manual-token requirement: Nepali at 20.22% and Telugu at 28.12% (marginally higher once the labeling issue above is accounted for). Nepali's margin is noticeably thinner than Telugu's, which makes sense given how much of Nepali's manual tier had to come from a narrower set of sources – Sangraha's Nepali subset is smaller relative to target, so the downloaded tier needed comparatively less padding to reach 500M tokens, but by the same token there was less manual text available overall to push that ratio further up. I'm treating 20.22% as tight rather than safely clear – if I collect any additional downloaded-tier data later, I'd want to pad Nepali's manual tier alongside it rather than let the ratio drift under 20%.

5 Train/Validation/Test Splits

Both languages use the same splitting procedure (`4_*_split_data.py`): the manual and downloaded tiers are shuffled and split independently (99% train / 0.5% val / 0.5% test, seed 42), then the resulting train/val/test pieces from each tier are concatenated and reshuffled. I did it this way specifically so the manual-token percentage stays roughly consistent across train/val/test rather than one tier accidentally clustering into a particular split – the per-split manual% figures in §4 (20.23/19.90/18.90 for Nepali, 28.14/27.12/25.37 for Telugu) confirm this held reasonably well, though the test-split manual share drifts a little lower than train in both languages, which is worth keeping in mind when interpreting val/test metrics later.

Table 4: Telugu – overall corpus statistics

Metric	Value
Total documents	983,814
Total words	284,703,353
Total characters	2,259,729,719
Total tokens	**547,479,659**
Fertility (avg. chars/token)	4.1275
UNK rate (val)	0.0000%
UNK rate (test)	0.0000%
Manual tokens	153,942,113 (28.12%)
Downloaded tokens	393,537,546 (71.88%)

Table 5: Telugu – per-split breakdown

Split	Docs	Tokens	Manual %
train	973,978	542,036,692	28.14%
val	4,918	2,800,728	27.12%
test	4,918	2,642,239	25.37%

6 Tokenizer Training

6.1 Methodology

I trained a SentencePiece BPE tokenizer from scratch for each language with byte-fallback enabled and `nmt_nfkc` normalization. Byte-fallback is the reason both languages report a flat 0.00% UNK rate at every vocabulary size I tested – any byte sequence can always be represented, so the usual "choose vocab size by UNK rate" heuristic from the brief doesn't actually discriminate between candidate sizes here. I chose the final vocabulary size on the bytes-per-token / embedding-parameter-cost trade-off instead, described below.

6.2 Vocabulary size ablation

Both ablation studies trained a fresh tokenizer at each candidate vocab size on a ∼10M-word random subset of that language's training split, then evaluated bytes-per-token and UNK rate on the full (untouched) validation and test sets.

6.3 Final vocabulary size: 12,000 (both languages)

The pattern in both tables is the same: doubling the vocabulary buys a shrinking return in compression (roughly +14% → +11% per doubling for Telugu, 8,000 → 16,000 → 32,000) while the embedding table's parameter cost still doubles every time. For a ∼25M-parameter model that matters a lot – at $d_{\text{model}} = 384$ (a representative width for this parameter budget), even the largest tested size, 32,000, eats close to half the budget on embeddings alone.

I settled on **12,000** for both languages, using the same five candidate sizes (8k/10k/12k/16k/32k) in both ablation studies. At $d_{\text{model}} = 384$ that's about 4.6M embedding parameters (∼18% of a 25M budget), leaving the large majority of the parameter budget for attention and feed-forward layers, while still capturing most of the compression benefit over the smallest tested size. This

Table 6: Telugu – per-source breakdown

Source	Docs	Tokens	% of total
Sangraha / AI4Bharat (downloaded)	775,498	388,434,297	70.95%
News websites (scraped)	204,605	143,957,746	26.29%
OCR archive (uncategorized tag)	169	5,103,249	0.93%
OCR – textbooks (manual)	92	5,024,117	0.92%
Literature websites (scraped)	1,082	3,041,087	0.56%
Wikipedia API	1,312	1,069,103	0.20%
Archive OCR	11	656,970	0.12%
Wiktionary API	1,013	162,230	0.03%
Web scrape (generic)	32	30,860	0.01%

Table 7: Nepali – vocabulary ablation (test set, ∼10M-word training subset)

Vocab size	Train time (s)	Bytes/token	Tokens/doc	UNK %
8,000	5.46	11.0473	470.69	0.0000
10,000	4.99	11.5354	450.77	0.0000
12,000	4.94	11.9046	436.79	0.0000
16,000	4.86	12.4527	417.57	0.0000
32,000	6.13	13.5524	383.68	0.0000

was a directly-measured ablation point for both languages, and using an identical tested set across both also removes vocabulary size as a confound when I compare Model H against Model L later in Phase 2/3.

One caveat worth stating plainly: since byte-fallback keeps UNK rate at exactly 0.00% regardless of vocabulary size, my selection here is really answering "how much parameter budget should the tokenizer's embedding table consume" rather than "at what vocabulary size does the tokenizer stop being able to represent the language" – those are different questions, and only the first one was actually decidable from this ablation.

6.4 Token-frequency statistics

Computed by tokenizing each language's ∼10M-word ablation training subset with the final 12,000-vocabulary tokenizer and counting token-ID frequencies.

Both languages show the same broad shape: essentially the whole vocabulary gets used (98.7–98.8% coverage) rather than sitting on a lot of dead entries, and the frequency distribution is heavily Zipfian – roughly 5% of the vocabulary accounts for half of all token occurrences in both languages. The single most frequent tokens in each language are punctuation and the most common grammatical suffixes/particles: for Nepali, the danda (।, the Devanagari sentence-final punctuation mark) and common postpositions (को, मा, का, ले); for Telugu, the full stop and comma dominate even more heavily (together over 6% of all tokens), followed by extremely common case markers and suffixes (లో, ను, లు, ని). That Telugu's top tokens skew more toward short 1–2 character morphological suffixes than Nepali's fits with Telugu being a more strongly agglutinative language – BPE ends up carving out a lot of short, highly reusable suffix pieces rather than the same volume of standalone short words.

Table 8: Telugu – vocabulary ablation (test set, ∼10M-word training subset)

Vocab size	Train time (s)	Bytes/token	Tokens/doc	UNK %
8,000	4.90	10.1486	582.40	0.0000
10,000	4.89	10.6176	556.68	0.0000
12,000	4.94	10.9941	537.61	0.0000
16,000	5.15	11.5718	510.77	0.0000
32,000	5.25	12.8723	459.17	0.0000

Table 9: Token-frequency summary (final 12k-vocab tokenizer, 10M-word sample)

Metric	Nepali	Telugu
Sample size (lines)	288,331	323,678
Total tokens produced	14,455,789	19,157,096
Unique token IDs used	11,854 / 12,000 (98.8%)	11,841 / 12,000 (98.7%)
Tokens needed for 50% of occurrences	618	766
Tokens needed for 90% of occurrences	5,841	6,403
Singleton tokens (freq. = 1 in sample)	12 (0.1%)	6 (0.1%)

6.5 Tokenization examples

Pulled from held-out validation-set documents, tokenized with each language's final 12,000-vocab model. An underscore (_) marks the start of a new word – this stands in for SentencePiece's own word-boundary meta-symbol, substituted here for typographic reliability across fonts.

Nepali

- **Original:** चीनमा दुई प्रहरीसहित 11 जनाको मृत्यु
  Tokens (7): _चीनमा _ दुई _ प्रहरी सहित _11 _जनाको _मृत्यु
- **Original:** 4 प्रतिशत आर्थिक वृद्धि हासिल गर्न कठिन !!
  Tokens (9): _4 _प्रतिशत _आर्थिक _वृद्धि _हासिल _गर्न _कठिन _! !
- **Original:** खोजी अभियान : चार दिनमा दुई सय 26 जना पक्राउ
  Tokens (10): _खोजी _अभियान _: _चार _दिनमा _दुई _सय _26 _जना _पक्राउ

Telugu

- **Original:** మహిళలకు ఉపాధి కల్పించేందుకు □సెట్విన్ శిక్షణ' : మంత్రి వాకిటి శ్రీహరి
  Tokens (15): _మహిళలకు _ఉపాధి _కల్ప ించేందుకు _□ సెట్ విన్ _శిక్షణ ' _: _మంత్రి _వా కి టి _శ్రీహరి
- **Original:** లంచం డబ్బుతో దొరికితే .. నో బెయిల్ .. ఓన్లీ జైల్
  Tokens (15): _లంచ ం _డబ్బు తో _దొరికి తే _.. _నో _బెయిల్ _.. _ఓ న్ లీ _జై ల్
- **Original:** ఏబీ వెంకటేశ్వరరావుని వేధించాలనే దురుద్దేశంతోనే క్రిమినల్ కేసు.. న్యాయవాది బి ఆదినారాయణరావు
  Tokens (23): _ఏ బీ _వెంకట ేశ్వరరావు ని _వేధ ించ ాలనే _దు రు ద్దేశ ంతోనే _క్రి మిన ల్ _కేసు .. _న్యాయవాది _బి _ఆద ిన ారాయణ రావు

A pattern worth noting: the Telugu examples above tokenize into noticeably more pieces per word than the Nepali ones for a comparable sentence length, which is consistent with the higher bytes-per-token figures Telugu showed throughout the ablation study (Table 8) even at the same vocabulary size – Telugu's richer agglutinative morphology means more distinct surface forms per root, so a fixed-size BPE vocabulary has to fall back to smaller sub-word pieces more often to cover the same amount of text.

6.6 Unknown-token rate

0.00% for both languages, on both val and test sets, at every vocabulary size tested. This is a direct consequence of `byte_fallback=True` in the SentencePiece training config – any input, including characters never seen during tokenizer training, decomposes into raw UTF-8 byte tokens rather than mapping to `<unk>`. This is expected behavior, not a data-quality result, and I want to be upfront that it shouldn't be read as evidence the corpora have unusually broad character coverage – it's a property of the tokenizer configuration.

7 Conclusion and Next Steps

Phase 1 is complete for both languages: cleaned, deduplicated, split corpora exceeding the ∼500M token target, both comfortably (Telugu) or narrowly (Nepali) clearing the ≥20% manual-data requirement, and a from-scratch SentencePiece BPE tokenizer trained and finalized at a 12,000-token vocabulary for each language, chosen on a parameter-budget argument appropriate to the project's ∼25M-parameter target model size.